Day 2

Documentation – Apex Collections, Triggers & Governor Limits

Name: Jinugu Nikitha

Topic: Apex Collections, Triggers & Governor Limits

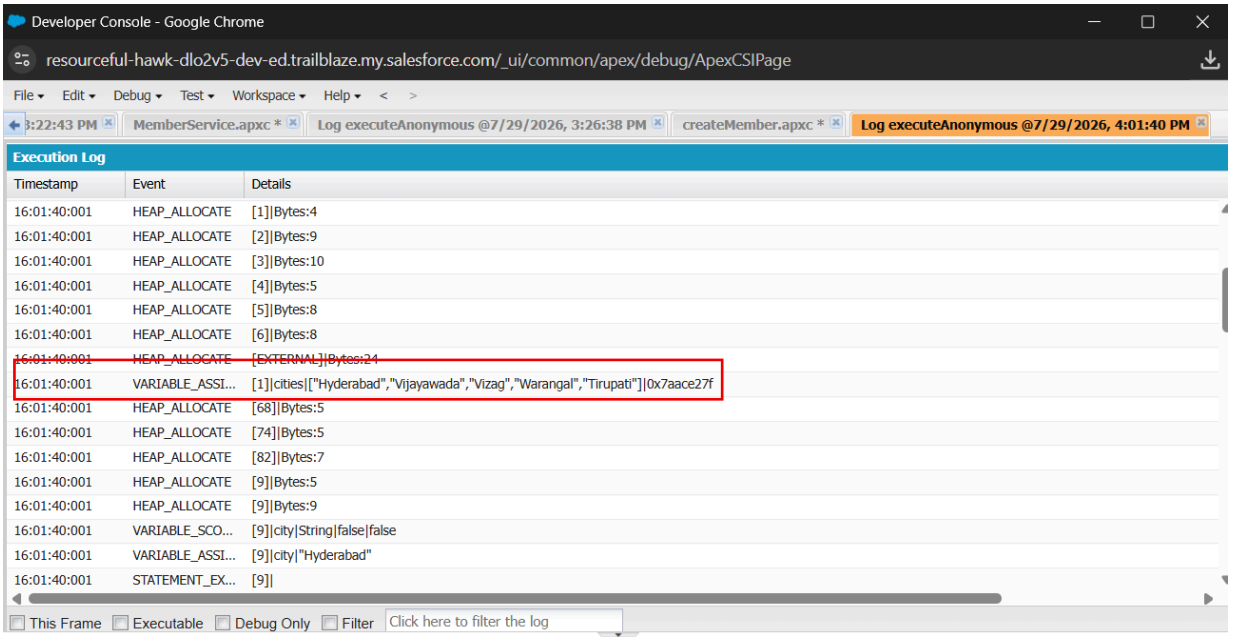
Objective:

The objective of Day 2 was to understand how Apex Triggers work, why Governor Limits exist in Salesforce, and how to write efficient and bulkified Apex code using Collections (List, Set, and Map). The assignment also focused on deciding when to use Validation Rules, Flows, and Triggers based on business requirements.

1. Apex Collections

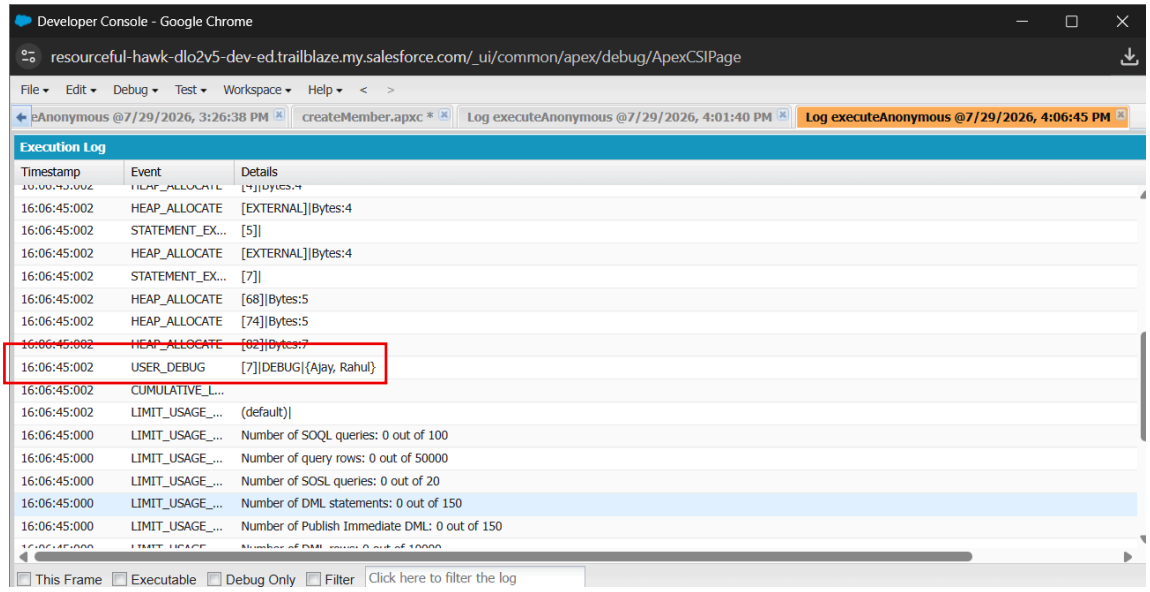
List:

- A List stores multiple records while preserving their order.
- Duplicate values are allowed.
- Used for processing multiple records returned from SOQL or Trigger.new.



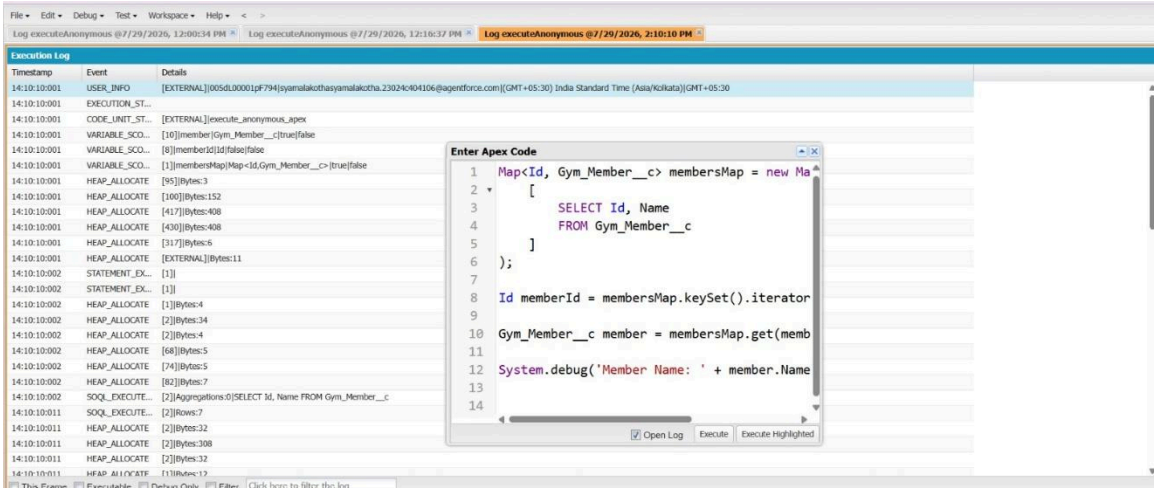
Set:

- A Set stores only unique values.
- Duplicate records are automatically removed.
- Mainly used for collecting unique record IDs before executing SOQL queries.



Map:

- A Map stores data as Key–Value pairs.
- In Salesforce, the key is usually the record Id.
- Used for quick retrieval of records without writing additional SOQL queries.



2. DML Operations

Learned basic Apex DML operations:

- Insert
- Update
- Delete

Example:

Members__c member = new Members__c();

member.Name = 'Mahesh';

insert member;

1. Governor Limits

Governor Limits are runtime restrictions that prevent Apex code from monopolizing shared Salesforce resources.

Some important limits are:

- Maximum **100 SOQL queries** in a synchronous transaction.
- Maximum **150 DML statements** in a transaction.
- Triggers execute on batches of **up to 200 records** at a time.

---

2. Bulkification

Bulkification is the process of writing Apex code that can process **multiple records efficiently** in a single transaction.

Best practices include:

- Process records using Trigger.new.
- Use collections (List, Set, Map).
- Perform SOQL queries outside loops.
- Perform DML operations outside loops.
- Query all required records at once using the IN clause.

Practical Activities Performed

Task 1: Created a Non-Bulkified Trigger

A trigger was intentionally written with a **SOQL query inside a for loop**.

This implementation was designed to demonstrate how inefficient code can exceed Salesforce Governor Limits when processing a large number of records.

Observation

- The trigger worked correctly for a few records.
- When hundreds of records were processed, Salesforce executed a SOQL query during every loop iteration.
- This caused the transaction to exceed the maximum SOQL query limit.

Task 2: Trigger Failure Due to Governor Limits

A bulk data load containing more than **200 records** was executed.

Expected Result

The transaction failed with an error similar to:

System.LimitException:

Too many SOQL queries: 101

This demonstrated why SOQL queries should never be placed inside loops.

Task 3: Bulkified the Trigger

The trigger was redesigned by:

- Moving the SOQL query outside the loop.
- Retrieving all required records using a single SOQL query.
- Using collections such as **Set** and **Map** to store and access records efficiently.

- Performing DML operations only once after processing all records.

Outcome

- Successfully processed **200+ records**.
- No Governor Limit exceptions occurred.
- Trigger performance improved significantly.

Block 3: Introduction to Asynchronous Apex using Future Methods

---

Objective

The objective of this block was to understand the fundamentals of **Asynchronous Apex** and learn how to execute Apex code in the background using **Future Methods**. The practical task involved creating a simple @future method that accepted a record Id, queried the corresponding record, updated one of its fields, and verified its execution through **Apex Jobs**. Salesforce recommends asynchronous processing for operations that do not need to complete while the user waits, such as long-running processes, callouts, or background updates.

Introduction

By default, Apex code runs **synchronously**, meaning each statement executes immediately and the user waits until the entire transaction is complete.

Asynchronous Apex allows specific operations to run **in the background**, enabling users to continue working without waiting for long-running tasks to finish. This approach improves user experience and helps optimize resource utilization.

Concepts Learned

1. Synchronous Processing

In synchronous processing:

- Code executes immediately.
- The user waits until execution is complete.
- Suitable for simple and quick operations.

---

2. Asynchronous Processing

In asynchronous processing:

- Tasks are placed in a queue.
- Salesforce executes them when system resources become available.
- The user does not wait for completion.
- Ideal for long-running operations and external web service callouts.

---

3. Future Method

A Future Method is an Apex method annotated with @future that executes asynchronously.

Characteristics:

- Must be a **static** method.
- Must return **void**.
- Can accept only **primitive data types** or collections of primitive data types as parameters.
- Cannot accept sObjects directly as parameters.

Practical Activity Performed

Task: Update a Gym Member Record Asynchronously

A Future Method was created to:

- Accept a **Member record Id**.

- Retrieve the corresponding **Member__c** record.
- Update the **Verified__c** checkbox field.
- Save the changes using a DML update operation.

The method was executed through **Execute Anonymous**, passing the Member record Id as the parameter.

Verification

After execution:

- The **Verified** field was successfully updated.
- The execution was confirmed by navigating to:

Setup

↓

Apex Jobs

The job appeared with the status **Completed**, confirming that it executed asynchronously rather than synchronously.

Key Observations

- Future Methods execute in the background.
- They improve application responsiveness by preventing long-running operations from blocking users.
- Only primitive parameters (or collections of primitive parameters) are supported.
- Future Methods are commonly used for:
    - External web service callouts.
    - Background processing.

- Resource-intensive operations.
- Avoiding mixed DML scenarios.

Comparison of Asynchronous Techniques

Future Method	Queueable Apex
Uses @future annotation	Implements the Queueable interface
Accepts only primitive parameters	Supports non-primitive member variables such as sObjects
No job ID is returned to the caller	Returns an AsyncApexJob ID using System.enqueueJob()
Cannot chain jobs	Supports chaining of queueable jobs
Older asynchronous approach	Recommended for most new asynchronous implementations

---

Why Batch Apex?

Batch Apex is used when processing **very large volumes of records** that exceed the limits of a single transaction. Instead of processing everything at once, Salesforce divides the work into smaller batches, making large-scale data processing more efficient and reliable.

Applications

Asynchronous Apex is commonly used in:

- Background record updates.
- External API integrations.
- Email notifications.
- Large data processing.

- Scheduled system operations.
- Batch data maintenance.